Laporan Praktikum 3: Klasifikasi dengan Ensemble Method dan Penanganan Class Imbalance pada Dataset Credit Card Fraud Detection

Zico Diego Rio Ramadhonny (5054251023)

1. Pendahuluan

Hasil utama: Random Forest tanpa penanganan khusus (baseline) sudah paling seimbang, dengan F1-score makro 0,934 dan AUC-ROC 0,946 pada data uji. Tidak ada teknik penanganan class imbalance yang mengungguli F1-score makro baseline; teknik-teknik tersebut terutama menggeser kompromi antara precision dan recall pada kelas penipuan.

Laporan ini merangkum Praktikum 3 (Classification dengan Ensemble Method dan Class Imbalance) oleh Zico Diego Rio Ramadhonny (5054251023). Tugasnya adalah mendeteksi transaksi kartu kredit yang merupakan penipuan, pada data yang sangat tidak seimbang.

- **Dataset:** Credit Card Fraud Detection (mlg-ulb, Kaggle), 284.807 transaksi dan 31 kolom.
- **Fitur:** `Time`, `V1` sampai `V28` (fitur anonim), dan `Amount`. Target adalah `Class` (0 = transaksi sah, 1 = penipuan).
- **Model:** `RandomForestClassifier(random_state=42)` pada semua skenario, sehingga perbedaan hasil berasal dari cara menangani ketidakseimbangan kelas.
- **Skenario:** delapan, yaitu baseline, cost-sensitive, tiga teknik oversampling (SMOTE, RandomOverSampler, ADASYN), dan tiga teknik undersampling (RandomUnderSampler, NearMiss, TomekLinks).

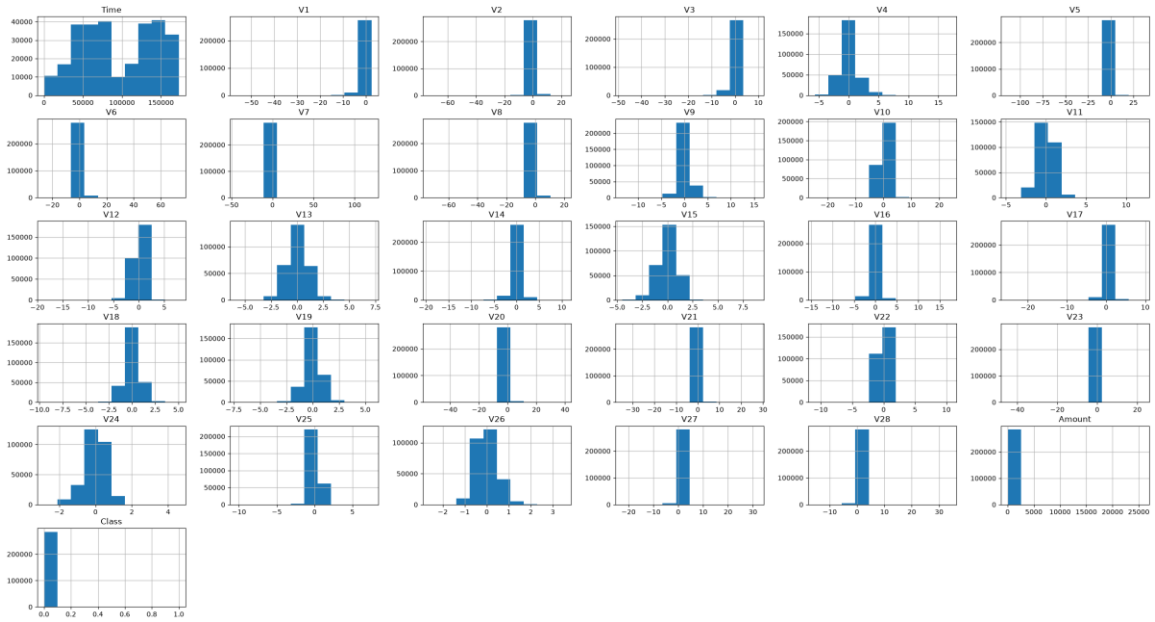
2. Hasil EDA

Data bersih dan sepenuhnya numerik. Sinyal terkuat terhadap penipuan datang dari V17, V14, V12, dan V10, sedangkan `Time` dan `Amount` hampir tidak berkorelasi linear dengan target.

2.1 Struktur dan statistik data

- **Ukuran:** 284.807 baris dan 31 kolom (30 kolom `float64` dan `Class` bertipe `int64`), memori 67,4 MB.
- **Missing value:** tidak ada; semua kolom terisi penuh (284.807 non-null).
- **`Time`:** berkisar 0 sampai 172.792 (sekitar 48 jam jika satuannya detik, sesuai deskripsi dataset), rata-rata 94.813,86.
- **`V1` sampai `V28`:** rata-rata sangat dekat nol (orde 10^{-15}) dengan simpangan baku antara 0,33 (V28) dan 1,96 (V1). Ini konsisten dengan fitur anonim hasil transformasi PCA. Rentang nilainya lebar, misalnya V5 dari -113,74 sampai 34,80.
- **`Amount`:** rata-rata 88,35, median 22,00, kuartil ketiga 77,17, tetapi maksimum 25.691,16 dan simpangan baku 250,12. Distribusinya sangat miring ke kanan.
- **`Class`:** rata-rata 0,001727, artinya hanya sekitar 0,17% transaksi bernilai 1.

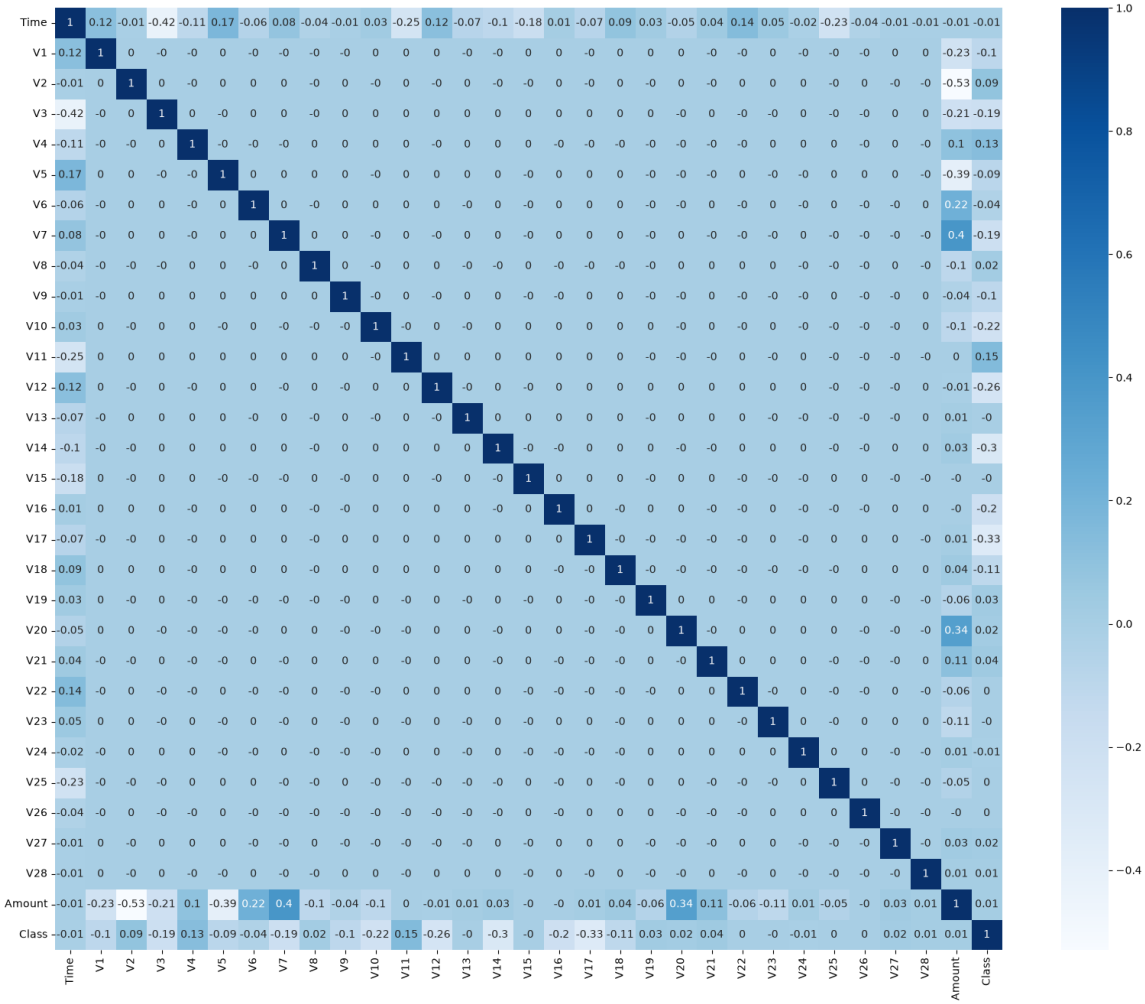
2.2 Distribusi fitur



Histogram (10 bin per fitur) memperlihatkan tiga pola:

1. `Time` memiliki dua puncak dengan lembah di sekitar 100.000, yang kemungkinan mencerminkan pola harian transaksi.
2. `Amount` sangat condong: hampir seluruh transaksi berada di bin pertama, sementara nilai ekstrem mencapai lebih dari 25.000.
3. Banyak fitur `V` (misalnya V1, V2, V3, V5, V6, V7, V8, V20, V21, V23) terpusat tajam di sekitar nol dengan ekor panjang akibat outlier, sedangkan V4, V9, V11, V13, V15, V19, V24, dan V26 lebih menyerupai lonceng.

2.3 Korelasi antar fitur dan terhadap target



Fitur V1 sampai V28 praktis tidak saling berkorelasi (nilai sekitar 0), sehingga tidak ada masalah multikolinearitas di antara komponen. Korelasi yang menonjol hanya melibatkan `Time` dan `Amount`: `Amount` dengan V2 (-0,53), V7 (0,40), V5 (-0,39), dan V20 (0,34), serta `Time` dengan V3 (-0,42).

Korelasi terhadap target (`Class`) diurutkan menurut nilai mutlak:

Peringkat	Fitur	Korelasi dengan Class
1	V17	-0,326
2	V14	-0,303
3	V12	-0,261
4	V10	-0,217
5	V16	-0,197
6	V3	-0,193

Peringkat	Fitur	Korelasi dengan Class
7	V7	-0,187
8	V11	0,155
9	V4	0,133

Fitur lainnya memiliki korelasi di bawah 0,12 secara mutlak. Yang terlemah adalah V22 (0,0008), V23 (-0,0027), V25 (0,0033), V15 (-0,0042), V26 (0,0045), V13 (-0,0046), `Amount` (0,0056), dan V24 (-0,0072); `Time` bernilai -0,0123. Semua korelasi linear terhadap target lemah (maksimum 0,33), wajar karena penipuan hanya 0,17% data. Model non-linear seperti Random Forest tetap dapat memanfaatkan pola yang tidak tertangkap korelasi linear.

3. Praproses Data

Praproses hanya mencakup pemisahan data secara stratified dan standardisasi fitur, karena data tidak memiliki nilai hilang maupun kolom kategorikal. Proporsi penipuan terjaga sama di data latih dan data uji (0,173%).

3.1 Langkah-langkah

1. **Pisahkan fitur dan target.** `X` berisi 30 kolom (semua kolom kecuali `Class`) dan `y` berisi `Class`.
2. **Bagi data latih dan uji** dengan `train_test_split(random_state=42, stratify=y)`. Karena `test_size` tidak diatur, porsi default 25% data uji digunakan. Opsi `stratify=y` menjaga proporsi kelas di kedua bagian.
3. **Standardisasi fitur** dengan `StandardScaler`. Scaler di-`fit` hanya pada data latih, lalu `transform` diterapkan pada data latih dan data uji, sehingga informasi data uji tidak bocor ke proses pelatihan.

Tidak ada imputasi (tidak ada missing value), tidak ada encoding (semua fitur numerik), serta tidak ada penghapusan outlier atau transformasi khusus pada `Amount`.

3.2 Hasil

Data	Jumlah baris	Transaksi sah (0)	Penipuan (1)	Proporsi penipuan
Latih	213.605	213.236	369	0,173%
Uji	71.202	71.079	123	0,173%

Setelah standardisasi, setiap fitur pada data latih memiliki rata-rata 0 dan simpangan baku 1, sehingga `Time` (hingga 172.792) dan `Amount` (hingga 25.691) tidak lagi mendominasi fitur `V`. Pada data uji, nilai hanya mendekati 0 dan 1 karena memakai parameter data latih. Random Forest sendiri tidak peka terhadap skala fitur; standardisasi di sini terutama berguna bagi teknik sampling berbasis jarak (SMOTE, ADASYN, NearMiss, dan TomekLinks) yang menghitung tetangga terdekat.

4. Distribusi Kelas Awal

Data sangat tidak seimbang: hanya 492 dari 284.807 transaksi (0,173%) adalah penipuan, sehingga transaksi sah sekitar 578 kali lebih banyak.

Kelas	Arti	Jumlah	Proporsi
0	Transaksi sah	284.315	99,827%
1	Penipuan	492	0,173%

Akibatnya, model yang selalu menebak "sah" sudah mencapai akurasi sekitar 99,83% tanpa mendeteksi satu penipuan pun. Karena itu akurasi tidak cukup untuk menilai model, dan evaluasi pada bagian 6 memakai precision, recall, F1-score, dan AUC-ROC. Selain itu, jumlah contoh penipuan yang sedikit (369 pada data latih) membuat model cenderung mengabaikan kelas minoritas jika data dibiarkan apa adanya.

5. Penanganan Class Imbalance

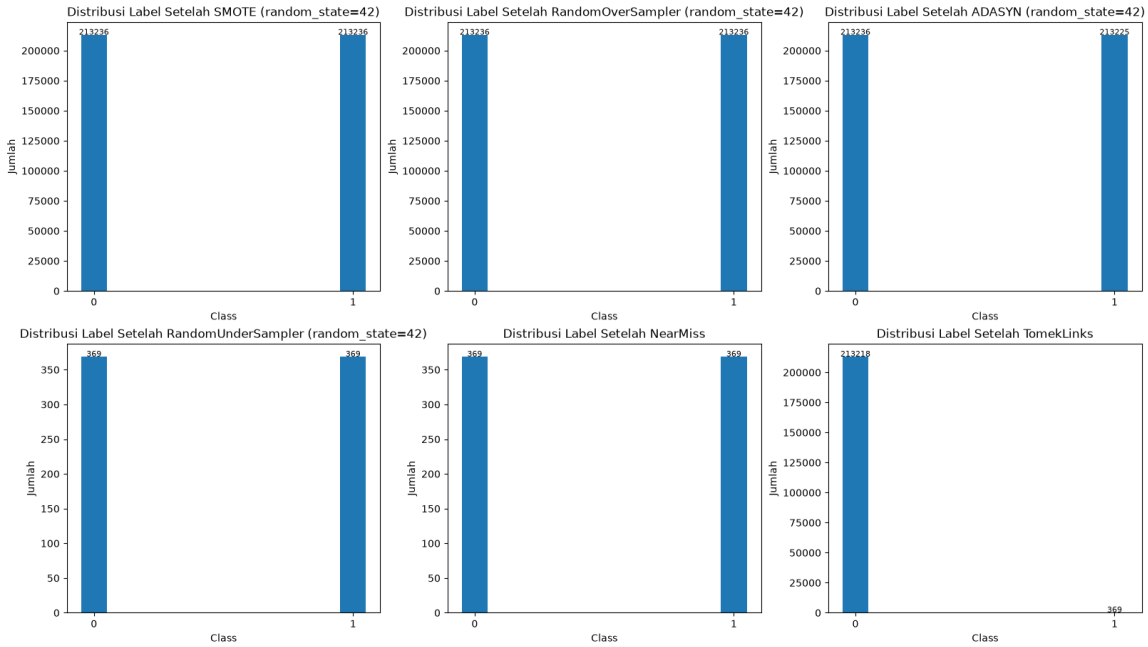
Seluruh teknik hanya diterapkan pada data latih (setelah pembagian dan standardisasi), sedangkan data uji dibiarkan pada distribusi aslinya (123 penipuan dari 71.202 transaksi). Dengan begitu, evaluasi mencerminkan kondisi nyata dan tidak terdistorsi data sintetis. Tiga teknik oversampling dan tiga undersampling meratakan kelas pada tingkat yang berbeda, sedangkan cost-sensitive learning tidak mengubah data sama sekali.

5.1 Teknik yang digunakan

Metode	Jenis	Cara kerja	Data latih sesudah	Sah (0)	Penipuan (1)
Baseline	Tanpa penanganan	Random Forest pada data asli	213.605	213.236	369
Cost-sensitive	Pembobotan kelas	`class_weight` = 1 − proporsi kelas, yaitu 0,0017 untuk kelas 0 dan 0,9983 untuk kelas 1 (rasio sekitar 578:1)	213.605	213.236	369
SMOTE	Oversampling	Membuat sampel penipuan sintetis lewat interpolasi	426.472	213.236	213.236

Metode	Jenis	Cara kerja	Data latih sesudah	Sah (0)	Penipuan (1)
		antar tetangga			
RandomOverSampler	Oversampling	Menggandakan sampel penipuan secara acak	426.472	213.236	213.236
ADASYN	Oversampling	Seperti SMOTE, lebih banyak sampel sintetis di area yang sulit dipelajari	426.461	213.236	213.225
RandomUnderSampler	Undersampling	Membuang transaksi sah secara acak	738	369	369
NearMiss	Undersampling	Mempertahankan transaksi sah yang paling dekat dengan kelas penipuan	738	369	369
TomekLinks	Undersampling (pembersihan)	Membuang transaksi sah yang berpasangan dengan penipuan sebagai tetangga terdekat	213.587	213.218	369

5.2 Distribusi kelas setelah penanganan



- **Oversampling** menaikkan kelas penipuan menjadi 213.236 (SMOTE dan RandomOverSampler) sehingga rasio 1:1. ADASYN menghasilkan 213.225, selisih 11 sampel, karena jumlah sampel sintetisnya ditentukan oleh tingkat kesulitan tiap sampel. Ukuran data latih menjadi dua kali lipat (sekitar 426 ribu).
- **RandomUnderSampler dan NearMiss** menyeimbangkan menjadi 369 : 369, tetapi menyusutkan data latih dari 213.605 menjadi 738 baris (turun 99,65%), sehingga hampir seluruh informasi transaksi sah terbuang.
- **TomekLinks** hanya membuang 18 transaksi sah, jadi kelas tetap sangat tidak seimbang (369 : 213.218). Teknik ini membersihkan batas antar kelas, bukan menyeimbangkan jumlahnya.
- **Cost-sensitive** tidak mengubah jumlah data; ketidakseimbangan ditangani lewat bobot, sehingga kesalahan pada kelas penipuan dihitung jauh lebih mahal saat pohon dibangun.

Dampak setiap teknik terhadap kinerja model dibahas pada bagian berikut.

6. Evaluasi Model

Baseline memperoleh F1-score makro tertinggi (0,9341), sedangkan RandomUnderSampler mencapai AUC-ROC dan recall tertinggi (0,9749 dan 0,9352) dengan precision yang runtuh. NearMiss gagal total. Semua model dievaluasi pada 71.202 transaksi uji yang berisi 123 penipuan.

6.1 Metrik pada data uji

Metode	Precision	Recall	F1-score	AUC-ROC	Akurasi
Baseline	0,9713	0,9024	0,9341	0,9455	0,99958
Cost-sensitive	0,9698	0,8821	0,9214	0,9531	0,99951

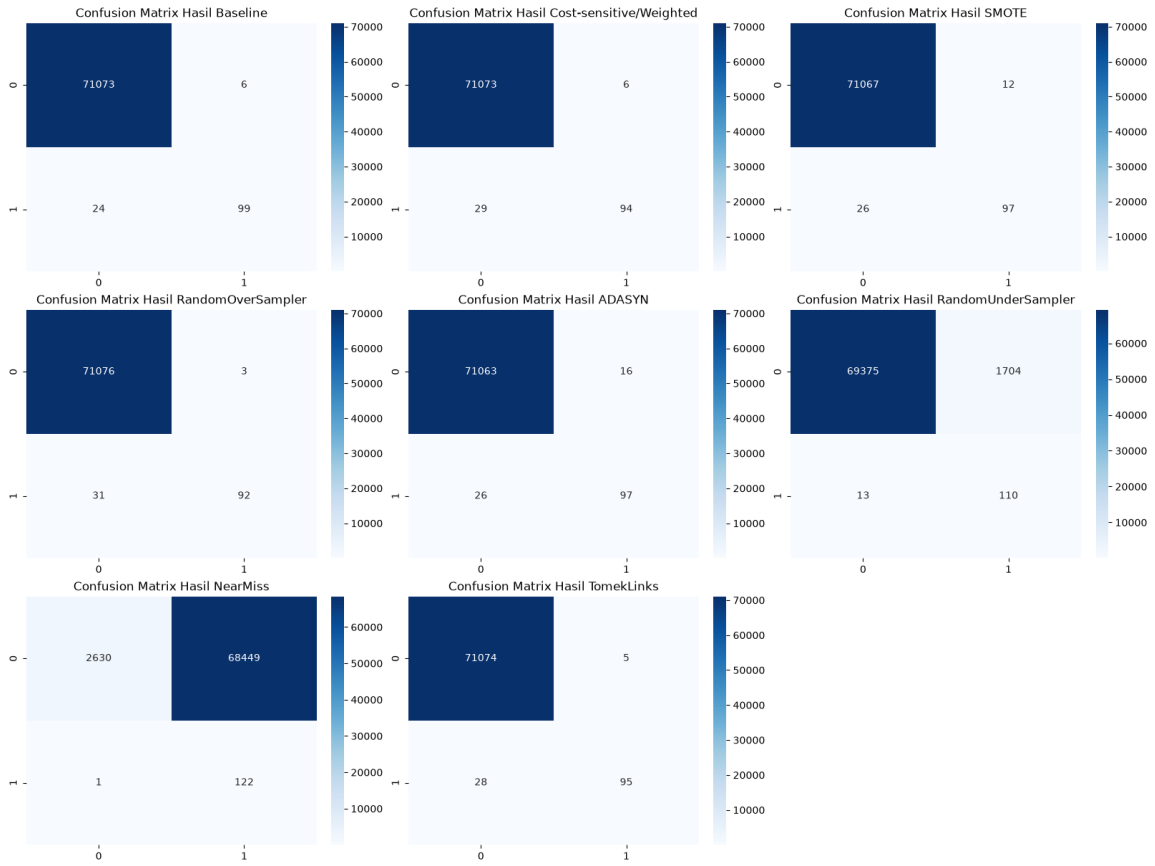
Metode	Precision	Recall	F1-score	AUC-ROC	Akurasi
SMOTE	0,9448	0,8942	0,9180	0,9646	0,99947
RandomOverSampler	0,9840	0,8740	0,9219	0,9493	0,99952
ADASYN	0,9290	0,8942	0,9109	0,9656	0,99941
RandomUnderSampler	0,5302	0,9352	0,5507	0,9749	0,97589
NearMiss	0,5007	0,5144	0,0375	0,9033	0,03865
TomekLinks	0,9748	0,8861	0,9259	0,9457	0,99954

Precision, recall, dan F1-score di atas adalah rata-rata makro dari kedua kelas, sehingga angkanya didominasi kelas sah yang hampir selalu benar. AUC-ROC dihitung dari probabilitas kelas penipuan. Akurasi nyaris identik (0,9994 sampai 0,9996) pada enam metode pertama sehingga tidak membedakan model; hanya RandomUnderSampler dan NearMiss yang turun.

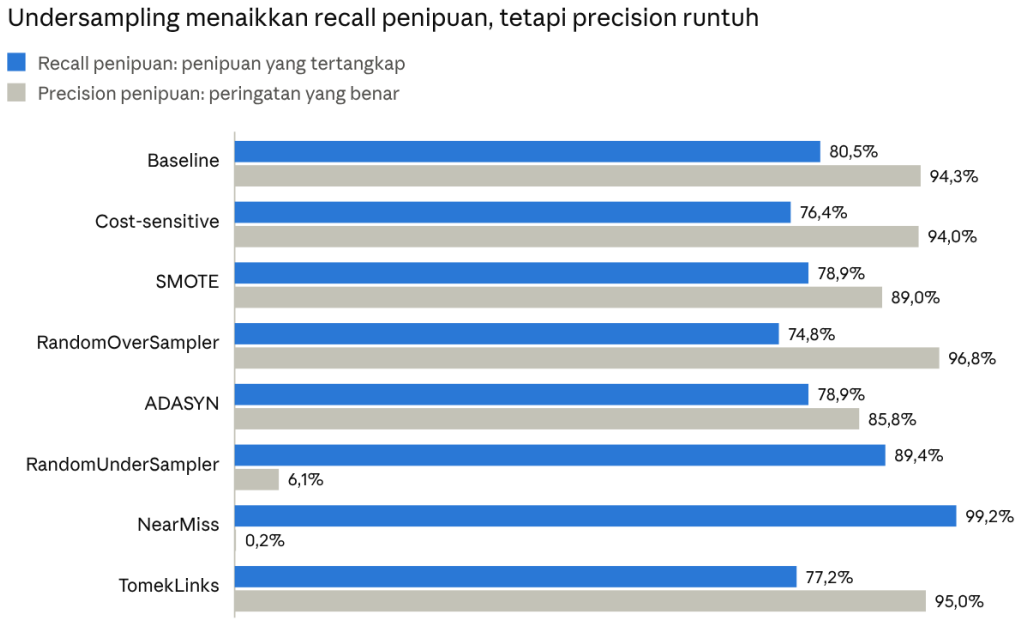
6.2 Confusion matrix kelas penipuan

Angka berikut berasal dari hasil `confusion_matrix` pada notebook (data uji: 123 penipuan dan 71.079 transaksi sah). Baris pada heatmap adalah kelas aktual dan kolom adalah kelas prediksi.

Metode	Penipuan terdeteksi (TP)	Penipuan lolos (FN)	Salah alarm (FP)	Sah benar (TN)	F1-score penipuan
Baseline	99	24	6	71.073	0,868
Cost-sensitive	94	29	6	71.073	0,843
SMOTE	97	26	12	71.067	0,836
RandomOverSampler	92	31	3	71.076	0,844
ADASYN	97	26	16	71.063	0,822
RandomUnderSampler	110	13	1.704	69.375	0,114
NearMiss	122	1	68.449	2.630	0,004
TomekLinks	95	28	5	71.074	0,852



6.3 Precision dan recall kelas penipuan



Dihitung dari confusion matrix pada notebook (123 penipuan, 71.079 transaksi sah); lihat tabel 6.2

Enam metode lainnya berkumpul di recall 75 sampai 81% dan precision 86 sampai 97%; hanya undersampling yang bergeser jauh ke recall tinggi dengan precision rendah.

6.4 Pembahasan

1. **Baseline sulit dikalahkan.** Baseline menangkap 99 dari 123 penipuan (recall 80,5%) hanya dengan 6 salah alarm (precision 94,3%). Dari lima metode yang menjaga akurasi tetap tinggi, tidak ada yang menangkap lebih banyak penipuan (92 sampai 97 transaksi).
2. **Oversampling memperbaiki peringkat skor, bukan keputusan.** SMOTE dan ADASYN menaikkan AUC-ROC dari 0,9455 menjadi 0,9646 dan 0,9656, tetapi pada ambang 0,5 keduanya menangkap 97 penipuan dengan 12 dan 16 salah alarm (baseline 6). RandomOverSampler sebaliknya paling konservatif: precision penipuan 96,8% dengan 3 salah alarm, tetapi recall turun ke 74,8%. Penggandaan sampel yang sama kemungkinan membuat pohon terlalu yakin pada contoh yang sudah dilihat.
3. **Cost-sensitive menaikkan AUC-ROC (0,9531), tetapi recall turun.** Dengan bobot kelas sekitar 578:1, model hanya menangkap 94 penipuan. Pada percobaan ini pembobotan tidak memperbaiki keputusan pada ambang 0,5.
4. **TomekLinks hampir sama dengan baseline** (95 vs 99 terdeteksi, 5 vs 6 salah alarm, AUC-ROC 0,9457 vs 0,9455) karena hanya membuang 18 dari 213.236 transaksi sah.
5. **RandomUnderSampler menukar precision dengan recall.** Model menangkap 110 penipuan (recall 89,4%) dan memiliki AUC-ROC tertinggi, tetapi menghasilkan 1.704 salah alarm sehingga hanya sekitar 6% peringatan yang benar. Pelatihan hanya memakai 738 baris.
6. **NearMiss gagal.** Model menangkap 122 dari 123 penipuan, tetapi menolak 68.449 dari 71.079 transaksi sah (96,3%), dan akurasinya hanya 3,9%. Penyebab yang paling mungkin: NearMiss mempertahankan transaksi sah yang paling dekat dengan penipuan, sehingga data latih tidak lagi mewakili sebaran transaksi sah dan batas keputusan bergeser ke arah kelas sah.
7. **Pilihan model bergantung pada biaya kesalahan.** Jika meloloskan penipuan jauh lebih mahal daripada memeriksa transaksi sah, RandomUnderSampler dapat dipakai sebagai penyaring awal; jika biaya pemeriksaan tinggi, baseline atau TomekLinks lebih sesuai.

7. Kesimpulan dan Saran

Pada percobaan ini, Random Forest baseline tanpa penanganan imbalance memberi hasil paling seimbang, dan teknik penanganan imbalance hanya menggeser kompromi antara recall dan precision.

- Data sangat tidak seimbang (0,173% penipuan), sehingga akurasi (0,9994 sampai 0,9996 pada enam model) tidak membedakan model.
- Baseline menangkap 99 dari 123 penipuan dengan 6 salah alarm (F1-score makro 0,9341). Tidak ada teknik yang menambah jumlah penipuan terdeteksi tanpa menambah salah alarm.
- SMOTE, ADASYN, dan cost-sensitive menaikkan AUC-ROC (dari 0,9455 menjadi 0,9531 sampai 0,9656) tanpa memperbaiki keputusan pada ambang 0,5.
- RandomUnderSampler memberi recall 89,4% tetapi precision hanya 6,1%; NearMiss tidak layak dipakai (akurasi 3,9%).

Keterbatasan. Hasil berasal dari satu pembagian data dan satu `random_state`. Dengan hanya 123 penipuan di data uji, selisih satu transaksi mengubah recall sekitar 0,8 poin, sehingga perbedaan kecil

antara baseline dan teknik lain belum dapat dianggap bermakna. Hyperparameter Random Forest tidak disetel, ambang keputusan tetap 0,5, dan metrik yang dicetak notebook adalah rata-rata makro yang menyembunyikan kinerja kelas penipuan.

Saran. (1) Gunakan stratified cross-validation berulang agar perbandingan lebih stabil. (2) Laporkan precision, recall, dan F1-score per kelas serta PR-AUC (area di bawah kurva precision-recall) selain AUC-ROC. (3) Setel ambang keputusan, terutama untuk model hasil resampling yang AUC-ROC-nya naik tanpa perbaikan pada ambang 0,5. (4) Coba kombinasi sampling (misalnya SMOTE dengan pembersihan Tomek Links) dan penyetelan hyperparameter.